

WEB TASARIMINDA JAVASCRIPT'E GİRİŞ



İÇİNDEKİLER

- Web Tasarımında Javascript
- Javascript'in Temelleri
 - Değişkenler
 - Veri Türleri
 - Operatörler
- Basit Etkileşimler
 - Alt Başlık Buton ile Etkileşim
 - Form Doğrulama
 - Kullanıcıya Geri Bildirim



HEDEFLER

- JavaScript'in web tasarımındaki rolünü açıklayabileceksiniz.
- Değişken, veri tipleri ve operatörleri örneklerle tanıyabileceksiniz.
- HTML elemanlarını JavaScript ile etkileşimli hale getirebileceksiniz.
- Form doğrulama işlemlerini temel düzeyde gerçekleştirebileceksiniz.
- Kullanıcıya mesaj gösterme ve basit geri bildirim yöntemlerini uygulayabileceksiniz



Atatürk Üniversitesi
Açıköğretim Fakültesi

WEB TASARIMININ TEMELLERİ

Doç. Dr. Nurullah TAŞ

ÜNİTE 8

GİRİŞ

Web sayfalarının yalnızca bilgi sunan statik yapılar olmaktan çıkıp kullanıcıyla etkileşim kurabilen dinamik ortamlara dönüşmesinde JavaScript'in kritik bir rolü vardır. HTML, sayfanın temel iskeletini oluşturur. CSS ise bu iskelete görsel bir kimlik kazandırır. Ancak kullanıcıdan veri almak, formları kontrol etmek, menüleri açıp kapatmak, butonlara tepki vermek veya animasyonlar üretmek gibi işlevler söz konusu olduğunda JavaScript devreye girer. Bu nedenle JavaScript, modern web tasarımında vazgeçilmez bir teknoloji olarak kabul edilmektedir (Flanagan, 2020).

JavaScript'in en önemli özelliklerinden biri, istemci tarafında (client-side) çalışmasıdır. Yani kodlar, doğrudan kullanıcının web tarayıcısı tarafından yürütülür. Bu durum, hem sunucuya olan yükü azaltır hem de kullanıcı deneyimini hızlandırır. Örneğin, bir formda eksik bilgi girildiğinde hatanın hemen ekranda gösterilmesi, sunucuya ek talep gönderilmeden yalnızca JavaScript sayesinde mümkündür (Mozilla Developer Network [MDN], 2023).

Bugün tüm modern web tarayıcıları JavaScript'i desteklemekte ve bu dil, sadece basit doğrulamalardan öteye geçerek gelişmiş uygulamaların temelini oluşturmaktadır. Kütüphane ve çerçeveler (framework) aracılığıyla JavaScript; mobil uygulamalardan oyun geliştirmeye, sunucu tarafı programlamadan masaüstü yazılımlarına kadar geniş bir kullanım alanına ulaşmıştır (Haverbeke, 2018). Ancak bu ünite de temel amaç, öğrencilerin JavaScript'in web tasarımındaki işlevini kavraması, basit etkileşimler geliştirmesi ve HTML ile CSS bilgilerini dinamik yapılarla bütünleştirebilmesidir. Bu ünite de, web geliştirme sürecinde JavaScript'in rolü ve temel yapısı ele alınacaktır. Öncelikle değişkenler, veri türleri ve operatörler gibi JavaScript'in temel kavramları açıklanmıştır. Ardından bu bilgiler kullanarak basit etkileşimlerin nasıl gerçekleştirileceği üzerinde durulmuştur. Butonlar aracılığıyla kullanıcı ile etkileşim kurma, formların doğruluğunu kontrol etme ve kullanıcıya geri bildirim sunma gibi örneklerle JavaScript'in dinamik özellikleri somutlaştırılmıştır. Böylece, bölüm sonunda okuyucular JavaScript'in web sayfalarını daha etkileşimli, işlevsel ve kullanıcı dostu hâle getirmedeki önemini kavramış olacaklardır.

WEB TASARIMINDA JAVASCRIPT

Web sayfaları, kullanıcıya yalnızca bilgi sunan statik bir yapıdan daha fazlasını sunmak zorundadır. Modern web tasarımında kullanıcı beklentileri; hızlı geri bildirim, etkileşimli içerikler ve kişiselleştirilmiş deneyimlerdir. JavaScript, bu beklentileri karşılamak için kullanılan temel teknolojilerden biridir. HTML sayfanın yapısını tanımlarken, CSS görsel düzeni oluşturur. JavaScript ise bu iki teknolojiyi tamamlayarak sayfaya hareket, dinamizm ve etkileşim kazandırır (Duckett, 2014).

JavaScript'in temel rolü, kullanıcı ile web sayfası arasındaki etkileşimi sağlamaktır. Örneğin, bir butona tıklanınca açılan menü, doldurulan formda hatalı girişin anında kullanıcıya bildirilmesi veya sayfa yeniden yüklenmeden veri güncellenmesi gibi işlevler JavaScript sayesinde mümkündür. Bu özellik hem

kullanıcı deneyimini iyileştirmekte hem de web uygulamalarının hızını artırmaktadır (MDN, 2023).

Bunun yanında JavaScript, yalnızca basit etkileşimler için değil; aynı zamanda gelişmiş web uygulamaları geliştirmek için de kullanılır. Kütüphaneler (ör. jQuery) ve çerçeveler (ör. React, Angular, Vue.js) sayesinde JavaScript, günümüzde tek sayfa uygulamaları (SPA), mobil uygulamalar ve hatta masaüstü yazılımlar için güçlü bir araç haline gelmiştir (Osmani, 2018). Dolayısıyla JavaScript'in rolü, web sayfalarına küçük işlevler eklemekten, karmaşık uygulamaların geliştirilmesine kadar geniş bir yelpazeyi kapsamaktadır.

Bu bağlamda öğrencilerin, JavaScript'in yalnızca teknik işlevlerini değil; aynı zamanda kullanıcı deneyimini geliştirmeye yönelik katkılarını da anlamaları önemlidir. Çünkü çağdaş web geliştirme anlayışı, yalnızca işleyen bir site değil, aynı zamanda kullanılabilir, erişilebilir ve etkileşimli bir ortam oluşturmayı hedeflemektedir.

JAVASCRIPT'İN TEMELLERİ

JavaScript öğrenmeye başlarken öncelikle dilin temel yapı taşlarını kavramak gerekir. Her programlama dilinde olduğu gibi JavaScript de belirli kurallar, sözdizimi (syntax) ve yapılar üzerine kuruludur. Bu yapılar sayesinde veriler saklanabilir, işlenebilir ve kullanıcı ile etkileşim kurulabilir. Web sayfasına etkileşim kazandırmak için kullanılan en küçük adım, değişken tanımlamak, uygun veri tiplerini seçmek ve bu veriler üzerinde işlemler yapmaktır. Dolayısıyla JavaScript'in temelleri, dilin ilerleyen kısımlarında kullanılacak daha karmaşık özelliklerin anlaşılması için bir başlangıç noktasıdır (Flanagan, 2020).

Temel kavramların öğrenilmesi, yalnızca teorik bilgi edinmek değil, aynı zamanda pratik uygulamalar yapabilmek açısından da önemlidir. Örneğin, bir öğrencinin adını ekranda göstermek, sayısal verilerle hesaplama yapmak veya doğru–yanlış ifadeleri kontrol etmek için değişkenler, veri tipleri ve operatörlerin bilinmesi gerekir. Bu kavramlar, JavaScript'in “nasıl çalıştığını” anlamamızı sağlar ve web sayfalarında gerçek anlamda dinamik içerikler geliştirmek için ilk basamak olur (Haverbeke, 2018).

Değişkenler

Programlamada *değişken* (variable), bilgisayarın hafızasında geçici olarak veri saklamak için kullanılan bir yapıdır. Bir web sayfasında kullanıcı adı, yaş veya hesaplanan bir değer gibi bilgiler değişkenlerde tutulur. JavaScript'te değişkenler sayesinde değerler daha sonra tekrar kullanılabilir, güncellenebilir veya işlenebilir. Bu nedenle değişkenler, JavaScript'in temel yapı taşlarından biridir (Duckett, 2014).

JavaScript'te değişken tanımlamak için üç temel anahtar sözcük kullanılır: *var*, *let* ve *const*. Eski sürümlerde yaygın olarak kullanılan *var*, günümüzde yerini büyük ölçüde *let* ve *const* ifadelerine bırakmıştır. *let*, değeri sonradan

değiştirilebilecek veriler için kullanılırken, const, sabit kalması gereken veriler için tercih edilir.

Örneğin, öğrencinin adı her zaman değişebileceği için let ile tanımlanabilir; ancak matematikte pi sayısı gibi sabit bir değer const ile saklanmalıdır. Bu ayrım hem programın güvenilirliğini hem de okunabilirliğini artırır (Flanagan, 2020).

```
let ad = "Ayşe";  
const pi = 3.14;  
var yas = 20;  
  
console.log(ad); // Ayşe  
console.log(pi); // 3.14  
console.log(yas); // 20
```

Veri Türleri

JavaScript'te değişkenler, programlama sürecinin en temel yapı taşlarından biridir. Değişkenler farklı türden bilgileri saklamak amacıyla kullanılır ve saklanan bu bilginin türü veri tipi (data type) olarak adlandırılır. Bir değişkenin hangi veri tipinde tanımlandığını bilmek, onun üzerinde hangi tür işlemlerin yapılabileceğini belirler. Örneğin, sayısal veri tipinde tanımlanan bir değişkenle matematiksel işlemler yapılabilirken, metin (string) veri tipiyle karakter dizileri üzerinde birleştirme, parçalama veya arama gibi işlemler gerçekleştirilebilir. Dolayısıyla veri tipleri, yalnızca kodun çalışabilirliğini değil, aynı zamanda programın verimliliğini, okunabilirliğini ve hatasız çalışmasını da doğrudan etkiler (Haverbeke, 2018). Bu nedenle, yazılım geliştiricilerin değişkenlerin türünü iyi kavraması, programlama sürecinde mantıksal hataları önlemenin ve etkili çözümler üretebilmenin ön koşuludur.

JavaScript'te en sık kullanılan ilkel (primitive) veri tipleri arasında Number (sayılar), String (metinler), Boolean (doğru–yanlış değerleri), Null ve Undefined yer almaktadır. Bunun yanı sıra, Object (nesne) veri tipi ise daha karmaşık yapıları kapsamakta ve diziler (Array) ya da fonksiyonlar gibi özel türleri içermektedir. Örneğin, bir öğrencinin not ortalaması number, adı string, dersten geçip geçmediği boolean ile ifade edilirken, derslerin listesi bir array, öğrencinin tüm bilgileri ise bir object içerisinde tutulabilir (Flanagan, 2020).

Bu veri tipleri programın akışını düzenlediği gibi, kullanıcıyla olan etkileşimin de doğru biçimde kurgulanmasına yardımcı olur. Hatalı veri tipi kullanımı, örneğin bir string üzerinde doğrudan matematiksel işlem yapılması, programda beklenmedik sonuçlara yol açabilir. Bu nedenle JavaScript'te dinamik tip sistemi önemli bir avantaj ve aynı zamanda potansiyel bir hata kaynağıdır: Değişkenlerin veri tipleri çalışma zamanında belirlenir, bu da geliştiriciye esneklik sağlarken dikkatli olunmadığında hata riskini artırır. JavaScript'te en sık kullanılan veri tipleri şunlardır:

Number (Sayılar): Sayısal değerleri saklamak için kullanılır. Bu veri tipi hem tam sayıları hem de ondalıklı sayı değerlerini kapsar ve matematiksel işlemler için uygundur.

Örneğin, yaş, not ortalaması veya ürün fiyatı gibi değerler number tipinde tanımlanır.

```
let yas = 20;  
let ortalama = 78.5;
```

String (Metinler): Çift veya tek tırnak içinde yazılan karakter dizilerini temsil eder. Kullanıcıya mesaj göstermek, veri tabanında metin depolamak ya da dinamik içerik oluşturmak için kullanılır. Ayrıca string metodları sayesinde metin üzerinde birleştirme, arama ve değiştirme işlemleri yapılabilir.

```
let ad = "Ayşe";  
let mesaj = 'Merhaba';
```

Boolean (Mantıksal değerler): Yalnızca iki değer alır: true (doğru) veya false (yanlış). Bu veri tipi koşullu ifadelerde kritik bir rol oynar ve program akışını yönlendirmek için sıklıkla kullanılır. Örneğin, bir öğrencinin dersten geçip geçmediği veya bir kullanıcının sisteme giriş yapıp yapmadığı boolean ile ifade edilebilir.

```
let gecis = true; // dersten geçti  
let bursluMu = false; // burslu değil
```

Undefined (Tanımsız): Bir değişken tanımlandığında ama değer ataması yapılmadığında otomatik olarak undefined olur. Bu, programda değişkenin hazırlandığını ancak henüz kullanılmadığını gösterir. Çoğu zaman, beklenen bir değer eksik olduğunu fark etmek için hata ayıklama sürecinde önemlidir.

```
let not;  
console.log(not); // undefined
```

Null (Boş): Bilinçli olarak boş bırakılmış değerleri temsil eder. Geliştirici tarafından atanır ve değişkenin şu anda bir değere sahip olmadığını göstermek için kullanılır. Bu özelliğiyle undefined'dan farklıdır çünkü undefined otomatik olarak oluşurken, null kasıtlı olarak atanır.

```
let telefon = null; // şu an için yok
```

Array (Dizi): Birden fazla veriyi tek bir yapı altında saklamak için kullanılır. Diziler, hem aynı türden hem de farklı türden verileri içerebilir, bu da onları oldukça esnek kılar. Genellikle listeler, öğrenci isimleri, ürün katalogları veya sayısal veri grupları dizilerle temsil edilir.

```
let dersler = ["Web", "Java", "Veritabanı"];
```

Object (Nesne): Daha karmaşık verileri özellik-değer çiftleri şeklinde saklamak için kullanılır. Gerçek dünyadaki varlıkların dijital temsili olarak düşünülebilir; örneğin bir öğrenci nesnesi adı, yaşı ve bölümü gibi özellikler içerir.



Number hesaplama yapmamızı, String metinleri saklamamızı, Boolean doğru–yanlış durumlarını göstermemizi, Array birden çok bilgiyi düzenlememizi sağlar. Object ise daha kapsamlı bilgileri tek bir yerde toplamamıza yardımcı olur.

Nesneler ayrıca fonksiyonlar da barındırabildiği için yazılımda güçlü bir modelleme aracıdır.

```
let ogrenci = { ad: "Ali", yas: 20, bolum: "Bilgisayar Programcılığı" };
```

Operatörler

JavaScript'te operatörler, değişkenler veya sabit değerler üzerinde işlem yapmamızı sağlayan semboller ya da özel karakterlerdir. Programlama dillerinde operatörler, yalnızca matematiksel hesaplamalar için değil, aynı zamanda karşılaştırma yapmak, mantıksal kontrolleri gerçekleştirmek, atamalar yapmak veya koşulları birleştirmek için de kullanılır. Matematikte alışık olduğumuz toplama (+), çıkarma (–), çarpma (*) ve bölme (/) işaretleri JavaScript'te de aynı şekilde çalışır. Ancak JavaScript, matematiksel işlemlerin ötesinde çok daha geniş bir yelpazede operatör desteği sunar. Bu nedenle, operatörlerin doğru kullanımı programlamada doğru sonuçlar elde etmenin temel koşuludur (Flanagan, 2020).

Aritmetik operatörler

JavaScript'te aritmetik operatörler, programlama sürecinde en temel işlemleri gerçekleştirmek için kullanılan yapılardır ve matematikte alışkın olduğumuz sembollerin doğrudan karşılığıdır. Bu operatörler sayesinde programcılar sayısal değerler üzerinde toplama, çıkarma, çarpma, bölme ve mod alma gibi işlemleri kolaylıkla yapabilirler. Aritmetik operatörlerin temel amacı, veriler üzerinde hesaplama yaparak anlamlı sonuçlar üretmektir. Örneğin bir öğrencinin not ortalamasını hesaplamak için toplama ve bölme operatörleri birlikte kullanılabilir, ya da bir alışveriş sepetindeki ürünlerin toplam fiyatı yine toplama operatörüyle elde edilebilir. JavaScript'in esnek yapısı sayesinde bu operatörler sadece sayısal değerlerle sınırlı kalmaz; toplama operatörü aynı zamanda string birleştirme (concatenation) işlevi de görür. Bu durum, kullanıcıya mesaj göstermek veya farklı parçaları bir araya getirerek yeni bir string oluşturmak için oldukça kullanışlıdır. Ancak bu çift yönlü kullanım, yanlış veri tipleriyle çalışıldığında beklenmedik sonuçlar doğurabilir. Örneğin `5 + "elma"` ifadesi sayısal bir işlem yapmak yerine `"5elma"` şeklinde string birleştirme sonucu üretir. Çarpma, bölme ve mod operatörleri ise yalnızca sayısal değerler üzerinde çalışır ve matematiksel doğruluk için kritik öneme sahiptir. Mod operatörü, özellikle algoritmalarda sık kullanılır; örneğin bir sayının çift mi tek mi olduğunu belirlemek için `% 2` ifadesinden yararlanılır. Aritmetik operatörler aynı zamanda döngüler, sayaçlar ve oyun programlamasında da yaygın biçimde kullanılır. Örneğin bir oyunda karakterin can puanının her darbe aldığı azalması çıkarma operatörüyle, kazanılan bonusların eklenmesi ise toplama operatörüyle yapılır. Ayrıca büyük veri hesaplamalarında veya finansal işlemlerde bu operatörlerin doğru kullanımı, programın güvenilirliği açısından hayati önem taşır. Kısacası, aritmetik operatörler JavaScript'in en temel taşlarından biri olup, günlük hayattaki matematiksel mantığın yazılım ortamına aktarılmasını sağlar.

Özetle bu operatörler matematik işlemleri yapmamızı sağlar.

- + → Toplama

- - → Çıkarma
- * → Çarpma
- / → Bölme
- % → Kalanı bulma

```
let elma = 10;  
let armut = 3;  
  
console.log(elma + armut); // 13  
console.log(elma - armut); // 7  
console.log(elma * armut); // 30  
console.log(elma / armut); // 3.33...  
console.log(elma % armut); // 1
```

Atama operatörleri

Atama operatörleri, JavaScript'te bir değişkene değer yüklemek ya da mevcut değerini güncellemek için kullanılan sembollerdir. En basit haliyle = operatörü, sağ tarafta yer alan değeri sol tarafta tanımlanmış değişkene atar. Ancak programlamada sadece başlangıç değeri vermek yeterli değildir; değişkenlerin süreç boyunca güncellenmesi gerekir. İşte bu noktada farklı atama operatörleri devreye girer. +=, -=, *=, /=, %= gibi türevler, bir değişkenin mevcut değeri üzerinden işlem yaparak sonucu tekrar aynı değişkene atar. Örneğin bir öğrencinin puanı her doğru cevapta 10 artıyorsa puan += 10 ifadesiyle bu artış kolayca sağlanabilir. Benzer şekilde, bir oyuncunun kalan can puanı her hasar aldığı anda belirli bir oranda azaltılabilir. Bu operatörler yalnızca sayısal işlemlerde değil, string değerlerde de kullanılabilir; örneğin mesaj += " Hoş geldiniz!" ifadesi, mevcut stringin sonuna yeni bir ifade ekler. Atama operatörleri, özellikle döngülerde ve tekrarlayan işlemlerde yazılımı daha okunabilir ve kısa hale getirir. Uzun formdaki puan = puan + 10 ifadesi yerine puan += 10 kullanmak hem daha pratiktir hem de kodun amacını daha net gösterir. Bunun yanı sıra atama operatörlerinin yanlış kullanımında program hataları ortaya çıkabilir. Örneğin = operatörü ile == karşılaştırma operatörünün karıştırılması sık görülen bir hatadır; bu durumda geliştirici değer atamak isterken yanlışlıkla karşılaştırma yapabilir veya tam tersi yaşanabilir. Profesyonel uygulamalarda bu ayrımın doğru yapılması kritik önemdedir. Ayrıca atama operatörleri, değişkenin türünü değiştirmez; yani sayı üzerinde yapılan bir işlem yine sayı olarak kalır, string üzerinde yapılan işlem ise string olarak devam eder. Bu özellik, JavaScript'in dinamik tip sisteminde bazen avantaj bazen de dezavantaj oluşturabilir. Sonuç olarak, atama operatörleri bir programın akışını yönetmek ve değişkenlerin güncel kalmasını sağlamak için temel araçlardan biridir.

Özetle bu operatörler bir değişkene değer verirken kullanılır.

- = → Değer atar

- += → Üzerine ekler
- -= → Üzerinden çıkarır

```
let puan = 50;

puan += 10; // puan = 60

puan -= 5; // puan = 55
```

Karşılaştırma operatörleri

Karşılaştırma operatörleri, iki değeri kıyaslamak için kullanılır ve her zaman bir Boolean sonucu döndürür: yani ya true (doğru) ya da false (yanlış). Bu özellik, koşullu ifadelerde karar vermek için büyük önem taşır. Örneğin bir öğrencinin geçip geçmediğini belirlemek için ortalama ≥ 50 ifadesi kullanılabilir; bu ifade öğrencinin ortalaması 50 veya daha yüksekse true, aksi halde false döndürecektir. JavaScript'te karşılaştırma operatörleri farklı türlerden değerleri de kıyaslayabilir, ancak bu durumda dikkatli olmak gerekir. $==$ operatörü sadece değere bakarken, $===$ hem değeri hem de veri tipini kontrol eder. Örneğin $5 == "5"$ ifadesi true dönerken, $5 === "5"$ false döner çünkü biri sayı diğeri string'dir. Modern programlamada genellikle $===$ kullanımı önerilir çünkü daha güvenilir sonuçlar üretir. Bunun yanında $!=$ ve $!==$ operatörleri eşit olmama durumlarını kontrol eder. Karşılaştırma operatörleri yalnızca sayılar için değil, stringler üzerinde de çalışır. Alfabetik karşılaştırmalarda, karakterlerin Unicode değerlerine göre bir sıralama yapılır. Örneğin $"a" < "b"$ ifadesi true döner. Bu özellik, sözlük sıralaması ya da isim listelerinin düzenlenmesi gibi işlemlerde kullanılabilir. Ayrıca büyük veri kümelerinde filtreleme ve arama fonksiyonlarının temelinde de karşılaştırma operatörleri vardır. Bir e-ticaret sitesinde fiyatı 100 TL'den düşük ürünleri listelemek için $urun.fiyat < 100$ ifadesi kullanılabilir. Ancak JavaScript'in gevşek tip sistemi, bazen beklenmedik sonuçlar doğurabilir; örneğin $null == undefined$ ifadesi true dönerken, $null === undefined$ false döner. Bu yüzden programcıların hangi karşılaştırma türünü kullandığına dikkat etmesi gerekir. Karşılaştırma operatörleri, kullanıcı doğrulama, form kontrolleri, erişim izinleri ve oyun mekaniği gibi birçok senaryoda kritik rol oynar. Doğru kullanıldığında yazılımın güvenilirliğini artırır, yanlış kullanıldığında ise ciddi mantık hatalarına yol açabilir.

Özetle bu operatörler iki değeri karşılaştırır, sonuç **true (doğru)** veya **false (yanlış)** çıkar.

- $==$ → Eşit mi? (sadece değere bakar)
- $===$ → Tam eşit mi? (hem değere hem veri tipine bakar)
- $!=$ → Eşit değil mi?
- $>$ → Büyük mü?
- $<$ → Küçük mü?
- $\geq$ → Büyük veya eşit mi?

- <= → Küçük veya eşit mi?

```
let x = 5;  
let y = "5";  
console.log(x == y); // true (değer aynı)  
console.log(x === y); // false (biri sayı, biri metin)  
console.log(x > 3); // true  
console.log(x < 3); // false
```

Mantıksal operatörler

Mantıksal operatörler, birden fazla koşulu bir arada değerlendirmek için kullanılan güçlü araçlardır ve genellikle if-else yapılarıyla birlikte çalışırlar. JavaScript'te üç temel mantıksal operatör vardır: && (ve), || (veya) ve ! (değil). && operatörü, yalnızca her iki koşul da doğru olduğunda true döner, aksi halde false verir. Bu, sıkı kontrol gerektiren durumlarda kullanışlıdır; örneğin bir kullanıcının hem giriş yapmış hem de admin rolüne sahip olması gerektiğinde `girisYapti && adminMi` ifadesi kullanılır. || operatörü ise daha esnek bir kontrol sağlar; koşullardan en az biri doğruysa sonuç true döner. Bu, alternatif senaryolarda sık kullanılır; örneğin bir öğrencinin başarılı sayılması için ya final notunun yüksek ya da ödev notlarının ortalamasının yeterli olması gibi durumlarda. ! operatörü ise koşulun tersini alır; true olan bir ifadeyi false, false olanı ise true yapar. Mantıksal operatörlerin gücü, birden fazla koşulu tek bir ifade içinde birleştirebilmesidir. Bu sayede karmaşık kontrol yapıları daha basit ve okunabilir hale gelir. Örneğin `yas > 18 && (ehliyetVar || veliIzniVar)` ifadesi, 18 yaşından büyük olanların doğrudan, küçüklerin ise veli izni veya ehliyeti varsa işlem yapabilmesini sağlar. Bu tür kombinasyonlar gerçek hayattaki karar verme süreçlerini yazılım dünyasına taşır. Ayrıca mantıksal operatörler performans açısından da önemlidir çünkü JavaScript'te kısa devre (short-circuit) mantığı uygulanır. Örneğin `false && ifade` ifadesinde ikinci kısım hiç değerlendirilmez, çünkü sonucun zaten false olacağı bellidir. Bu özellik kodun daha verimli çalışmasını sağlar. Mantıksal operatörler güvenlik denetimlerinde, kullanıcı doğrulama süreçlerinde, oyun mekaniği tasarımlarında ve veri filtreleme işlemlerinde yoğun olarak kullanılır. Yanlış kullanıldığında program akışı tamamen bozulabilir, bu nedenle geliştiricilerin mantıksal ifadeleri dikkatle kurmaları gerekir. Özetle, mantıksal operatörler yazılımda mantığın, koşul kombinasyonlarının ve karar mekanizmalarının bel kemiğini oluşturur.

Özetle bu operatörler koşulları birleştirmek için kullanılır.

- && → ve (her iki koşul da doğruysa)
- || → veya (koşullardan biri doğruysa)
- ! → değil (sonucu tersine çevirir)



Aritmetik operatörler sayılar üzerinde hesaplama yapar, atama operatörleri değişkenlere değer verir veya mevcut değeri günceller, karşılaştırma operatörleri iki değeri birbirleriyle kıyaslar, mantıksal operatörler ise birden fazla koşulu birleştirerek sonucun doğru ya da yanlış olmasını belirler.

```
let yas = 20;
console.log(yas > 18 && yas < 25); // true (her ikisi de doğru)
console.log(yas < 18 || yas > 30); // false (ikisi de yanlış)
console.log(!(yas > 18));           // false (doğru olan koşulu tersine çevirdi)
```

BASİT ETKİLEŞİMLER

JavaScript'in en önemli özelliklerinden biri, web sayfasını kullanıcı ile etkileşimli hale getirme gücüdür. HTML, sayfanın yapısal içeriğini; CSS ise bu içeriğin görsel düzenini tanımlar. Ancak bu iki teknoloji tek başına kullanıldığında sayfa durağan kalır ve kullanıcı sadece hazır verilere pasif olarak erişir. İşte tam bu noktada JavaScript devreye girerek web sayfalarına dinamizm ve etkileşim kazandırır. Örneğin bir düğmeye basıldığında ekrana bir uyarı mesajı çıkması, bir formun boş bırakıldığında kullanıcıya uyarı vermesi ya da ziyaretçiye doğrudan soru sorulması JavaScript sayesinde mümkündür. Bu tür uygulamalar web deneyimini yalnızca işlevsel değil aynı zamanda eğlenceli, akıcı ve kullanıcı dostu hale getirir (Duckett, 2014).

Etkileşimli web tasarımı, modern internetin temel gerekliliklerinden biridir. Kullanıcıların yalnızca bilgiye ulaşmaları değil, aynı zamanda bu bilgiyle doğrudan etkileşim kurmaları beklenmektedir. JavaScript ile geliştirilen olay yönetimi (event handling) mekanizması sayesinde, kullanıcıların yaptığı her hareket –örneğin fare tıklaması, klavyeden tuşa basma, bir öğenin üzerine gelme– sayfa üzerinde belirli bir tepki doğurabilir. Böylece kullanıcı, web sayfasının sadece izleyicisi değil, aktif bir katılımcısı haline gelir. Bu yaklaşım, kullanıcı deneyimini (UX) zenginleştirir ve sitenin daha uzun süre ziyaret edilmesini sağlar.

Ayrıca JavaScript'in sunduğu dinamik özellikler, yalnızca görsel efektlerle sınırlı değildir. Form doğrulama, veri girişlerinin kontrol edilmesi, kullanıcıya kişisel öneriler sunulması, canlı saat ve tarih göstergeleri veya sayfa yenilenmeden sunucu ile veri alışverişi yapılması gibi işlevler de JavaScript'in katkıları arasındadır. Özellikle AJAX teknolojisi, web sayfalarının daha hızlı ve kesintisiz çalışmasına imkân vererek kullanıcıların bekleme süresini azaltır. Bu da etkileşimin sürekliliğini sağlar.

Sonuç olarak JavaScript, web teknolojileri arasında köprü işlevi görerek HTML ve CSS'in sınırlı yapısını aşar ve sayfalara etkileşim katmanını ekler. Kullanıcıların ihtiyaçlarına anlık yanıt veren, daha dinamik ve kişisel deneyimler sunan bu özellik, günümüzün dijital dünyasında web sitelerinin vazgeçilmez bir bileşeni haline gelmiştir. JavaScript olmadan web yalnızca statik belgelerden

oluşurken, JavaScript sayesinde web yaşayan, kullanıcıyla iletişim kuran ve sürekli güncellenebilen bir platforma dönüşmüştür.

Buton ile Etkileşim

Web sayfalarında kullanıcı ile etkileşim kurmayı sağlayan en temel öğelerden biri butonlardır. Butonlar, kullanıcıların belirli bir eylemi başlatmalarına imkân tanır ve genellikle form gönderme, sayfa üzerinde değişiklik yapma, içerik yükleme ya da görsel-işitsel bir unsur çalıştırma gibi işlevlerde kullanılır. Günümüzde modern web uygulamalarında butonlar yalnızca görsel bir öğe olarak değil, aynı zamanda kullanıcı deneyiminin (UX) merkezinde yer alan etkileşim noktaları olarak değerlendirilir. Bir web sitesinde gezinirken karşılaşılan “Kaydet”, “Gönder”, “Renk Değiştir” veya “Sepete Ekle” gibi butonlar, kullanıcıların ihtiyaç duyduğu işlemleri kolay, hızlı ve anlaşılır biçimde gerçekleştirmesine yardımcı olur.

Butonların temel görsel tanımı HTML ile yapılır; örneğin <button> etiketi, sayfa üzerinde tıklanabilir bir öğe oluşturur. Ancak HTML'nin işlevi yalnızca bu butonu görüntülemekle sınırlıdır. Yani, HTML bir butonun yerleşimini ve etiketini tanımlar, fakat butonun bir işlem yapabilmesi için tek başına yeterli değildir. Bir butona tıklandığında belirli bir eylemin gerçekleşmesi, yani görselin ötesinde bir işlevsellik kazanması için mutlaka JavaScript gibi bir programlama dili ile desteklenmesi gerekir. Bu durum, HTML'in yapıyı, CSS'in görünümü, JavaScript'in ise davranışı ve etkileşimi kontrol ettiği üçlü mimari yaklaşımın önemli bir örneğidir.

JavaScript, butonları dinamik hale getirmek için olaylar (events) kavramını kullanır. Bu mekanizma, kullanıcının sayfa üzerindeki hareketlerine (örneğin tıklama, fareyle üzerine gelme, çift tıklama veya klavye etkileşimi) karşılık verilmesini sağlar. Butonlarla ilgili en yaygın kullanılan olaylardan biri onclick olayıdır. Kullanıcı butona tıkladığında bu olay tetiklenir ve tanımlanmış kod parçası çalıştırılır. Böylece, pasif bir görsel öğe olan buton, aktif ve işlevsel bir arayüze dönüşür. Aşağıdaki örnekte, bir butona tıklandığında ekrana bir uyarı mesajı çıkarılmaktadır:

```
<button onclick="alert('Merhaba!')">Tıkla</button>
```

Bu kod çalıştırıldığında kullanıcı butona bastığında ekranda küçük bir pencere açılır ve “Merhaba!” mesajı görünür. Görüldüğü gibi, yalnızca birkaç satır JavaScript kodu ile sayfaya etkileşim kazandırmak mümkündür.

Butonlarla yapılan etkileşimler yalnızca mesaj göstermekle sınırlı değildir. Bir butona tıklandığında sayfanın arka plan rengini değiştirmek, bir yazının rengini farklı yapmak ya da bir formu kontrol etmek de mümkündür. Örneğin, aşağıdaki kod parçasında butona basıldığında sayfanın arka plan rengi maviye dönüşmektedir:

```
<button onclick="document.body.style.backgroundColor='lightblue'">  
Arka Planı Değiştir  
</button>
```

Bu örnek, JavaScript'in butonlar aracılığıyla sayfanın görünümünü ve davranışını değiştirebildiğini göstermektedir. Sonuç olarak, butonlarla yapılan etkileşimler kullanıcı deneyimini artırmakta, sayfayı daha dinamik ve ilgi çekici hale getirmektedir.

Form Doğrulama

Formlar, web sayfalarında kullanıcıdan bilgi toplamak için kullanılan en yaygın araçlardan biridir. İletişim sayfalarındaki ad-soyad ve e-posta alanlarından, e-ticaret sitelerindeki ödeme formlarına, eğitim platformlarındaki kayıt ekranlarından devlet kurumlarının çevrim içi başvuru sistemlerine kadar çok geniş bir kullanım alanına sahiptirler. Kullanıcıların web ile etkileşiminin merkezinde çoğu zaman formlar yer almaktadır. Ancak formların sağlıklı çalışabilmesi için yalnızca görsel olarak doğru tasarlanması yeterli değildir. Kullanıcıların form alanlarını eksik, hatalı ya da uyumsuz verilerle doldurması sık görülen bir durumdur. Örneğin e-posta yerine rastgele bir metin yazılması, yaş alanına harf girilmesi veya zorunlu alanların boş bırakılması sistemin doğru çalışmasını engelleyebilir.

Bu tür durumlarda, yalnızca sunucu tarafında hata mesajı göstermek kullanıcı deneyimini olumsuz etkiler. Çünkü kullanıcı formu gönderdikten sonra hatayı görür ve tekrar geri dönüp düzeltmek zorunda kalır. İşte bu noktada JavaScript, web geliştirme sürecine büyük bir katkı sunar. JavaScript sayesinde hatalar anında tespit edilip kullanıcıya gösterilebilir. Örneğin, kullanıcı "Ad" alanını boş bıraktığında formun gönderilmesine izin verilmez ve ekranda uyarı mesajı çıkar. Bu yöntem, hem yanlış veri girişinin önüne geçilmesini sağlar hem de kullanıcıya daha hızlı, güvenilir ve yönlendirici bir deneyim sunar.

Ayrıca JavaScript ile yapılan form doğrulama, yalnızca eksik alanların doldurulmasını sağlamakla kalmaz; girilen verinin biçimsel doğruluğunu da kontrol edebilir. Örneğin, e-posta adreslerinde "@" işaretinin bulunup bulunmadığını, şifrelerin belirli bir uzunlukta olup olmadığını ya da telefon numarasının yalnızca sayılardan oluşup oluşmadığını kontrol etmek mümkündür. Bu sayede hem veri bütünlüğü korunur hem de sistemin işleme aldığı bilgilerin geçerliliği artırılır. Dolayısıyla, JavaScript form doğrulama süreci modern web uygulamalarında yalnızca bir ek özellik değil, neredeyse bir zorunluluk olarak görülmektedir.

Aşağıda basit bir form doğrulama örneği verilmiştir. Bu örnekte, kullanıcı “Ad” alanını boş bıraktığında ekrana uyarı mesajı çıkarılmaktadır:

```
<form onsubmit="return kontrolEt()">
  Ad: <input type="text" id="ad"><br><br>
  <input type="submit" value="Gönder">
</form>

<script>
function kontrolEt() {
  // "Ad" alanına girilen değeri alıyoruz
  let ad = document.getElementById("ad").value;
  // Eğer "Ad" alanı boş bırakılmışsa
  if (ad == "") {
    // Kullanıcıya uyarı mesajı gösteriyoruz
    alert("Ad alanı boş bırakılamaz!");
    // Formun gönderilmesini engelliyoruz
    return false;
  }
  // Eğer alan boş değilse, form normal şekilde gönderilir
}
</script>
```

Bu koddan öncelikle bir **form** tanımlanmıştır. Form gönderilmeden önce onsubmit="return kontrolEt()" ifadesi sayesinde JavaScript fonksiyonu devreye girmektedir. Fonksiyon, document.getElementById("ad").value koduyla kullanıcının “Ad” kutusuna yazdığı değeri alır. Eğer bu değer boşsa (ad == ""), alert() komutu ile ekrana “Ad alanı boş bırakılamaz!” mesajı çıkar ve return false ifadesi sayesinde formun gönderilmesi engellenir. Böylece kullanıcı eksik bilgi girdiğinde sistem hemen uyarı vererek hatayı düzeltmesini sağlar.

Kullanıcıya Geri Bildirim

Web sayfalarında kullanıcı ile etkileşimi güçlendiren en önemli unsurlardan biri de geri bildirim verilmesidir. Bir kullanıcı, yaptığı işlemin doğru ya da yanlış olduğunu, sistemin bu işlemi kabul edip etmediğini ya da bir sonraki adımda ne yapması gerektiğini anında öğrenmek ister. Eğer bu tür bilgilendirmeler sağlanmazsa, kullanıcı sistemin çalışmadığını, komutun işlenmediğini ya da hatalı bir işlem yaptığını düşünebilir. Bu durum yalnızca kullanıcı deneyimini (UX) zayıflatmakla kalmaz, aynı zamanda web sitesine duyulan güveni de azaltır. İşte bu noktada JavaScript, etkileşimli web tasarımına katkı sunan önemli araçlardan biri olarak devreye girer ve çeşitli fonksiyonlarıyla kullanıcıya anlık geri bildirim sağlama imkânı verir.

JavaScript'in sunduğu bu araçlardan en yaygın kullanılanları alert(), confirm() ve prompt() fonksiyonlarıdır.

- alert() fonksiyonu, kullanıcıya basit bir bilgilendirme mesajı göstermek için kullanılır. Örneğin bir form gönderildiğinde “Kayıt başarıyla tamamlandı” mesajı ya da bir hata durumunda “Eksik bilgi girdiniz” uyarısı ekranda küçük bir pencere olarak gösterilebilir. Bu pencere kullanıcı işlemine devam etmeden önce mutlaka kapatılmalıdır, dolayısıyla dikkat çekici bir geri bildirim sağlar.

- `confirm()` fonksiyonu, kullanıcının “Tamam” veya “İptal” seçeneklerinden birini seçmesine olanak tanır. Bu fonksiyon, kritik işlemlerde kullanıcıdan onay almak için idealdir. Örneğin, bir dosyanın silinmesi ya da bir hesabın kapatılması gibi geri dönüşü zor olan işlemlerde `confirm()` ile kullanıcıya “Bu işlemi yapmak istediğinizden emin misiniz?” sorusu yöneltilir. Kullanıcının seçimi, fonksiyonun döndürdüğü `true/false` değeriyle kontrol edilerek program akışı buna göre şekillenir.
- `prompt()` fonksiyonu, kullanıcıdan doğrudan metin girişi almayı sağlar. Kullanıcıya bir soru sorulur ve cevap olarak yazdığı metin program tarafından alınır. Örneğin, kullanıcıya “Adınızı giriniz:” şeklinde bir uyarı penceresi açılır ve girilen değer bir değişkende saklanarak programın sonraki adımlarında kullanılabilir. Bu yöntem, küçük çaplı veri toplama işlemlerinde ya da kişiselleştirilmiş deneyimler sunmak için tercih edilir.

Bu üç fonksiyon basit görünmekle birlikte, web sayfalarında kullanıcı ile doğrudan iletişim kurmanın en kolay yollarını sunar. Ancak modern web uygulamalarında bu fonksiyonların yerine çoğu zaman daha estetik ve kullanıcı dostu tasarımlar tercih edilmektedir. Örneğin Bootstrap veya SweetAlert gibi kütüphaneler, `alert()` fonksiyonunun sunduğu basit pencereleri çok daha şık ve özelleştirilebilir hale getirir. Buna rağmen, JavaScript'in temel düzeyde sağladığı bu fonksiyonlar, dilin etkileşim yeteneklerini kavramak ve kullanıcı geri bildiriminin önemini anlamak açısından kritik bir başlangıç noktasıdır.

Aşağıdaki örnekte bu üç fonksiyonun nasıl kullanıldığı gösterilmektedir:

Örnek



```

•<script>
•// Kullanıcıya basit bir bilgilendirme mesajı gösteriyoruz
•alert("Sitemize hoş geldiniz!");

•// Kullanıcıya evet/hayır sorusu soruyoruz
•let devam = confirm("Devam etmek istiyor musunuz?");

•// Eğer kullanıcı 'Tamam' derse
•if (devam) {
•alert("Devam ediliyor...");
•} else {
•alert("İşlem iptal edildi.");
•}

•// Kullanıcıdan isim bilgisi istiyoruz
•let ad = prompt("Lütfen adınızı giriniz:");

•// Eğer kullanıcı bir değer girdiyse
•if (ad != null && ad != "") {
•alert("Merhaba, " + ad + "!");
•}
•</script>

```

Bu örnekte önce alert() ile kullanıcıya bir hoş geldiniz mesajı gösterilmektedir. Daha sonra confirm() fonksiyonu ile kullanıcıya bir soru yöneltilmekte, cevaba göre farklı mesajlar ekrana çıkarılmaktadır. Son olarak prompt() fonksiyonu ile kullanıcıdan ad bilgisi alınmakta ve bu bilgi kullanılarak kişiselleştirilmiş bir mesaj gösterilmektedir.

Bu üç fonksiyon, özellikle başlangıç seviyesinde JavaScript öğrenenler için kullanıcı etkileşimini anlamayı kolaylaştırır. Ancak modern web uygulamalarında çoğunlukla daha gelişmiş ve tasarıma uygun geri bildirim yöntemleri tercih edilmektedir. Yine de bu temel fonksiyonlar, JavaScript'in kullanıcıyla nasıl iletişim kurduğunu göstermesi açısından önemlidir.

UYGULAMA ÖRNEKLERİ

JavaScript'in gücü, basit örneklerde çok daha kolay anlaşılabilir. Aşağıda yer alan uygulamalar, öğrencilerin hem önceki bölümlerde öğrenilen temel kavramları pekiştirmesine hem de JavaScript'in web sayfalarına nasıl canlılık kattığını

görmesine yardımcı olacaktır.



Örnek

•Örnek 1: Kullanıcıyı Selamlama

•Bu uygulamada kullanıcıdan adını girmesi istenir ve ardından ekranda kişisel bir selamlama mesajı gösterilir.

```
•<!DOCTYPE html>
•<html>
•<head>
•<title>Selamlama Örneği</title>
•</head>
•<body>
•<button onclick="selamla()">Adınızı Girin</button>

•<script>
•function selamla() {
•// prompt ile kullanıcıdan ad bilgisi alınıyor
•let ad = prompt("Lütfen adınızı giriniz:");

•// Kullanıcı bir şey yazmışsa ekrana mesaj gösteriliyor
•if (ad != null && ad != "") {
•alert("Merhaba, " + ad + "!");
•}
•}
•</script>
•</body>
•</html>
```

Bu örnekte JavaScript kullanılarak kullanıcı ile etkileşimli basit bir selamlama uygulaması gerçekleştirilmiştir. HTML yapısında bir buton tanımlanmış ve bu butona tıklandığında selamla() isimli fonksiyon çağrılacak şekilde onclick olayı eklenmiştir. Fonksiyon çalıştığında, JavaScript'in prompt() fonksiyonu devreye girer ve kullanıcıdan adını girmesi istenir. Kullanıcı bir değer girerse bu değer ad değişkenine aktarılır. Daha sonra koşul ifadesiyle, girilen değer boş olup olmadığı kontrol edilir; eğer kullanıcı bir isim yazmışsa ekranda alert() fonksiyonu aracılığıyla "Merhaba, [kullanıcı adı]!" mesajı görüntülenir. Böylece butona tıklayan kullanıcı, kişiselleştirilmiş bir selamlama mesajı ile karşılaşır. Bu örnek, JavaScript'in kullanıcıdan veri alma ve bu veriyi işleyerek geri bildirim verme yeteneğini basit ama etkili bir şekilde göstermektedir.

Örnek


Örnek 2: Basit Hesap Makinesi

Kullanıcıdan iki sayı alıp bunları toplayan küçük bir hesap makinesi uygulaması:

```

•<!DOCTYPE html>
•<html>
•<head>
•<title>Hesap Makinesi</title>
•</head>
•<body>
•<button onclick="topla()">Sayıları Topla</button>

•<script>
•function topla() {
•// Kullanıcıdan iki sayı alınıyor
•let sayi1 = prompt("Birinci sayıyı giriniz:");
•let sayi2 = prompt("İkinci sayıyı giriniz:");

•// Girilen değerler string olduğundan sayıya çevriliyor
•let toplam = Number(sayi1) + Number(sayi2);

•// Sonuç ekrana yazdırılıyor
•alert("Toplam: " + toplam);
•}
•</script>
•</body>
•</html>

```

Bu örnekte JavaScript kullanılarak kullanıcıdan alınan sayıları toplayan basit bir hesap makinesi uygulanmıştır. HTML sayfasında bir buton tanımlanmış ve bu butona tıklandığında topla() fonksiyonu çalışacak şekilde onclick olayı eklenmiştir. Fonksiyon çalıştığında, JavaScript'in prompt() fonksiyonu aracılığıyla kullanıcıdan iki farklı sayı girmesi istenir. prompt() fonksiyonu ile alınan veriler string (metin) türünde olduğu için, matematiksel işlem yapılabilmesi amacıyla Number() fonksiyonu kullanılarak sayıya dönüştürülür. Daha sonra bu iki sayı toplanarak toplam değişkenine atanır ve sonuç, alert() fonksiyonu aracılığıyla kullanıcıya "Toplam: [sonuç]" mesajı olarak gösterilir. Böylece kullanıcı, girdiği iki sayının toplamını ekranda anında görebilir. Bu örnek, JavaScript'in hem kullanıcıdan veri alma hem de veriyi işleyerek hesaplama yapma ve geri bildirim sağlama becerisini göstermektedir.

Örnek



•Örnek 3: Kayıt Formunda E-Posta Kontrolü

•Kullanıcıdan alınan e-posta adresinin doğru formatta olup olmadığını kontrol eden bir uygulama:

```

•<!DOCTYPE html>
•<html>
•<head>
•<title>E-posta Kontrolü</title>
•</head>
•<body>
•<form onsubmit="return kontrolEt()">
•E-posta: <input type="text" id="eposta">
•<input type="submit" value="Gönder">
•</form>

•<script>
•function kontrolEt() {
•// E-posta kutusundaki değer alınır
•let eposta = document.getElementById("eposta").value;

•// "@" sembolü içerip içermediği kontrol edilir
•if (eposta.indexOf("@") == -1) {
•alert("Lütfen geçerli bir e-posta adresi giriniz!");
•return false; // form gönderilmez
•}
•}
•</script>
•</body>
•</html>

```

Bu örnekte JavaScript kullanılarak basit bir e-posta doğrulama işlemi gerçekleştirilmiştir. HTML kısmında bir form oluşturulmuş ve form gönderilmeden önce kontrolEt() fonksiyonunun çalışması için onsubmit özelliği tanımlanmıştır. Kullanıcı "Gönder" butonuna bastığında, JavaScript fonksiyonu devreye girer ve document.getElementById("eposta").value ifadesiyle formdaki e-posta alanına girilen değer alınır. Ardından bu değer üzerinde indexOf("@") metodu ile kontrol yapılır; eğer girilen metin içerisinde "@" sembolü yoksa sonuç -1 döner. Bu durumda kullanıcıya alert() fonksiyonu aracılığıyla "Lütfen geçerli bir e-posta adresi giriniz!" uyarısı gösterilir ve return false ifadesi sayesinde formun gönderilmesi engellenir. Böylece kullanıcı hatalı bir giriş yaptığında sistem anında geri bildirim vererek doğru formatta veri girmesini sağlar. Bu örnek, JavaScript'in form doğrulama sürecinde nasıl kullanılabileceğini gösteren temel bir uygulamadır.



Özet

- Bu ünite JavaScript'in web tasarımındaki konumu ve işlevsel katkıları ayrıntılı biçimde ele alınmıştır. Günümüzde web teknolojileri HTML, CSS ve JavaScript olmak üzere üç temel bileşenden oluşmaktadır. HTML, bir web sayfasının yapısal iskeletini oluştururken, CSS bu yapıya görsel estetik ve düzen kazandırmaktadır. Ancak, kullanıcı ile sayfa arasındaki etkileşimi sağlayan ve statik yapıyı dinamik hâle getiren unsur JavaScript'tir. Dolayısıyla JavaScript, web tasarımını yalnızca görsel bir düzenleme süreci olmaktan çıkarak, işlevsel, etkileşimli ve kullanıcı odaklı bir yapıya dönüştürmektedir. Bu yönüyle JavaScript, modern web geliştirme sürecinin ayrılmaz bir parçası hâline gelmiştir.
- Ünitenin başlangıç bölümünde, JavaScript'in temel yapı taşları üzerinde durulmuştur. Programlama dillerinin en temel bileşenleri olan değişkenler, veri türleri ve operatörler, öğrencilerin programlama mantığını kavramaları açısından kritik öneme sahiptir. Değişkenler, verilerin bellekte saklanması ve gerektiğinde yeniden kullanılmasını sağlar. Veri türleri ise bu değişkenlerin hangi türde verilerle çalışabileceğini belirlemektedir. Örneğin sayılar (number), metinler (string), mantıksal değerler (boolean) ve nesneler (object) gibi türler, JavaScript'in esnek yapısını ortaya koymaktadır. Operatörler ise bu veriler üzerinde matematiksel, karşılaştırmalı veya mantıksal işlemler gerçekleştirmeyi mümkün kılmaktadır. Bu bölümde, değişken tanımlama yöntemleri (var, let, const), sayılar ve metinlerle işlem yapma, temel hesaplamalar ve string birleştirme gibi uygulamalar örneklerle açıklanmıştır. Böylelikle öğrencilerin, programlama dilinin mantığını kavrayarak daha karmaşık yapılara hazırlık yapmaları hedeflenmiştir.
- Ünitenin ikinci bölümünde JavaScript'in etkileşim boyutu ele alınmıştır. JavaScript'in en belirgin katkısı, kullanıcı davranışlarına anında yanıt verebilmesidir. Bir butona tıklandığında belirli bir işlemin gerçekleşmesi, form verilerinin doğruluğunun kontrol edilmesi ya da sayfa içeriğinin dinamik olarak güncellenmesi bu bağlamda öne çıkan örneklerdir. Bu noktada, JavaScript'in sunduğu alert(), confirm() ve prompt() gibi fonksiyonlar detaylı biçimde incelenmiştir. Bu fonksiyonlar, kullanıcıya doğrudan geri bildirim verme, onay alma veya veri girişi sağlama açısından önemli bir role sahiptir. Ayrıca, olay yönetimi (event handling) kavramı üzerinde durularak kullanıcı eylemlerine anında tepki verebilecek yapıların nasıl kurgulanacağı açıklanmıştır. Böylece, web sayfası ile kullanıcı arasındaki iletişimin nasıl daha etkin ve kişiselleştirilmiş hâle getirilebileceği vurgulanmıştır.
- Son bölümde ise kuramsal bilgilerin pekiştirilmesine yönelik çeşitli uygulama örneklerine yer verilmiştir. Kullanıcıyı selamlayan basit bir mesaj kutusu oluşturma, dört işlemden yalnızca toplama işlevini yerine getiren küçük bir hesap makinesi geliştirme ve e-posta adreslerinin geçerliliğini sınanan doğrulama uygulaması, bu kapsamda ele alınmıştır. Bu uygulamalar, öğrencilerin yalnızca dilin sözdizimini öğrenmekle kalmayıp aynı zamanda öğrendiklerini pratik senaryolarda uygulayabilmelerine imkân sağlamaktadır. Böylece, teorik bilginin günlük kullanım örnekleriyle pekiştirilmesi ve kalıcı öğrenmenin sağlanması amaçlanmıştır.
- Sonuç olarak, bu ünite JavaScript'in web sayfalarını sadece görsel açıdan değil, aynı zamanda işlevsel ve etkileşimsel açıdan da zenginleştiren bir teknoloji olduğunu vurgulamaktadır. JavaScript, kullanıcı deneyimini geliştirmeye yönelik sunduğu esnek ve güçlü olanaklar sayesinde web tasarımında merkezi bir konuma sahiptir. Ünite ele alınan temel kavramlar, öğrencilerin ilerleyen aşamalarda daha gelişmiş programlama konularını anlamaları için sağlam bir zemin oluşturmaktadır. Bu bağlamda, JavaScript'in öğrenilmesi, web geliştirme alanında ilerlemek isteyen herkes için stratejik bir başlangıç noktasıdır.

DEĞERLENDİRME SORULARI

- JavaScript'in temel görevi aşağıdakilerden hangisidir?
 - Web sayfasına dinamizm ve etkileşim kazandırmak
 - Web sayfasına görsel düzen kazandırmak
 - Web sayfasının yapısını tanımlamak
 - Veritabanı işlemlerini yürütmek
 - Sunucu taraflı programlama yapmak
- Aşağıdakilerden hangisi JavaScript'te değişken tanımlamak için kullanılan anahtar sözcüklerden biri değildir?
 - var
 - let
 - const
 - def
 - int
- Aşağıdaki kodun çıktısı ne olur?

```
let x = 10;  
let y = 3;  
console.log(x % y);
```

 - 3
 - 1
 - 0
 - 7
 - 10
- Kullanıcıdan bilgi almak için kullanılan JavaScript fonksiyonu hangisidir?
 - alert()
 - confirm()
 - prompt()
 - console.log()
 - document.write()
- Aşağıdaki karşılaştırma operatörlerinden hangisi hem değerin hem de veri tipinin eşitliğini kontrol eder?
 - ==
 - =
 - ===
 - !=
 - <=

6. Form doğrulamada return false; ifadesinin amacı nedir?
 - a) Formun her zaman gönderilmesini sağlamak
 - b) Kullanıcının formu tekrar doldurmasını sağlamak
 - c) Formdaki verileri silmek
 - d) Formu otomatik kaydetmek
 - e) Formun sunucuya gönderilmesini engellemek
7. Aşağıdakilerden hangisi JavaScript'te mantıksal operatördür?
 - a) +
 - b) &&
 - c) %
 - d) =
 - e) ++
8. Bir butona tıklandığında belirli bir işlemi tetikleyen olay (event) hangisidir?
 - a) Onsubmit
 - b) Onload
 - c) Onchange
 - d) Onkeypress
 - e) Onclick
9. Aşağıdaki veri tiplerinden hangisi JavaScript'e ait değildir?
 - a) Number
 - b) String
 - c) Boolean
 - d) Character
 - e) Undefined
10. Aşağıdaki kod ifadelerinden hangisi yanlıştır?
 - a) console.log() kullanıcıya uyarı mesajı kutusu açar.
 - b) alert() kullanıcıya bilgilendirme mesajı verir.
 - c) prompt() kullanıcıdan giriş alır.
 - d) confirm() doğru/yanlış türünde değer döndürür.
 - e) JavaScript'te değişkenler dinamik tipli olabilir.

Cevap Anahtarı

1.a, 2.d, 3.b, 4.c, 5.c, 6.e, 7.b, 8.e, 9.d, 10.a

YARARLANILAN KAYNAKLAR

Duckett, J. (2014). JavaScript and jQuery: Interactive front-end web development. Wiley.

Flanagan, D. (2020). JavaScript: The definitive guide (7th ed.). O'Reilly Media.

Haverbeke, M. (2018). Eloquent JavaScript: A modern introduction to programming (3rd ed.). No Starch Press.

Mozilla Developer Network (MDN). (2023). JavaScript guide. Mozilla Foundation.
<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>

Osmani, A. (2018). Learning JavaScript design patterns. O'Reilly Media.